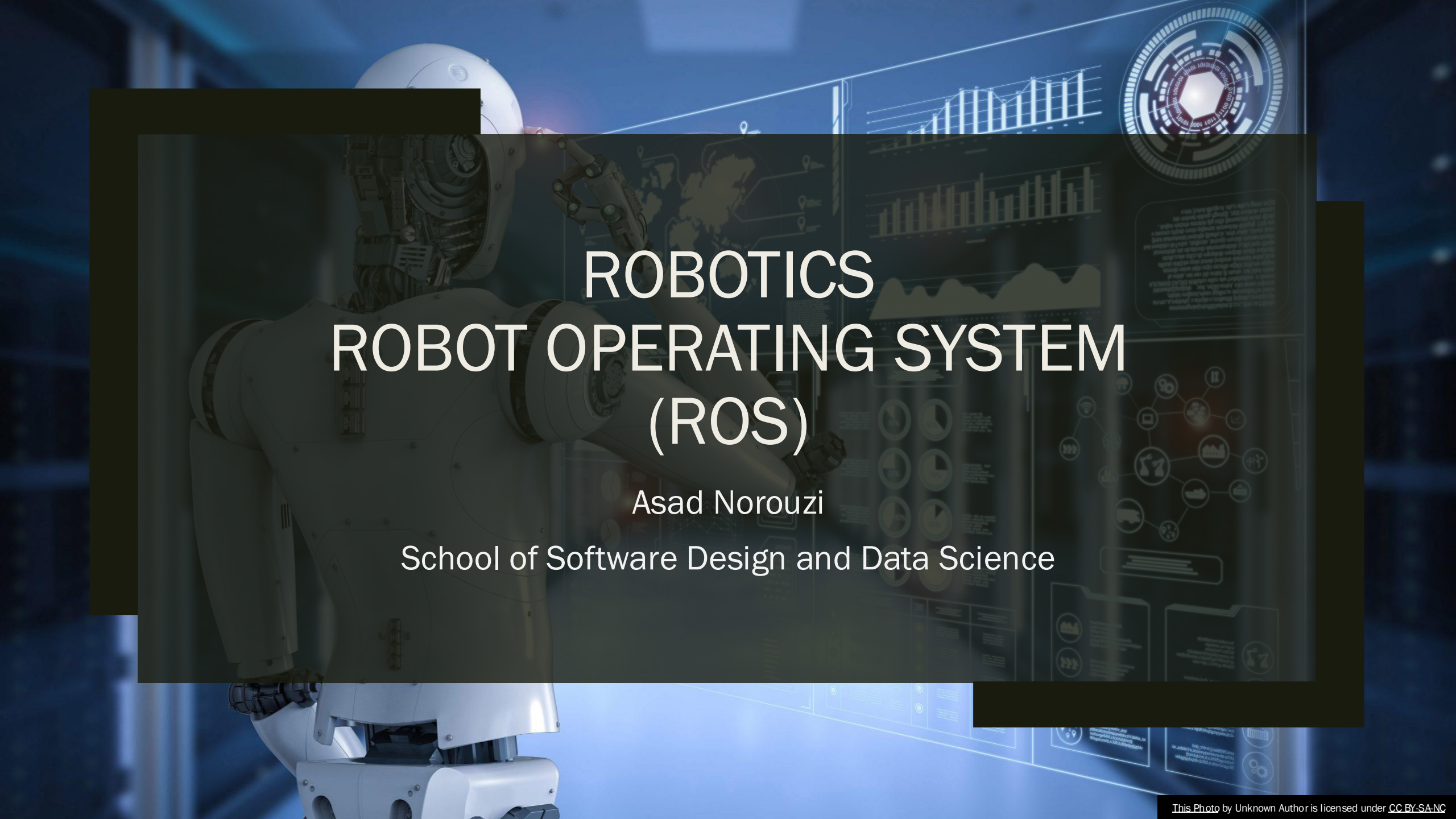


# ROBOTICS
# ROBOT OPERATING SYSTEM (ROS)

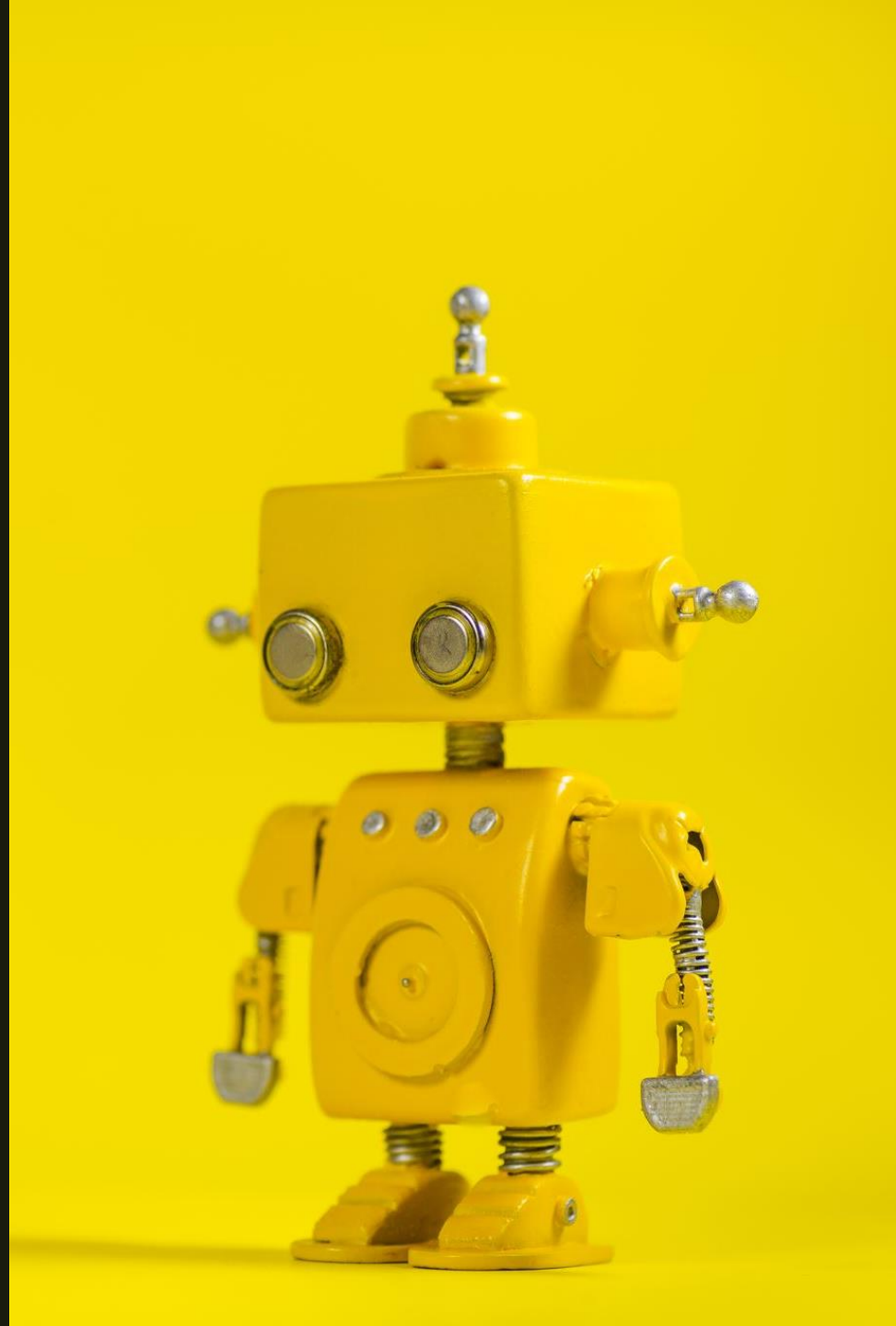
Asad Norouzi

School of Software Design and Data Science

This Photo by Unknown Author is licensed under CC BY-SA-NC

# Agenda

- Introduction to ROS
- ROS 1 vs ROS 2
- ROS Architecture
- Setting Up ROS
- ROS Packages and Tools
- ROS Communication
- ROS Messages
- Creating and Compiling ROS Nodes
- ROS Launch Files
- Simulation in ROS
- Building A Simple ROS Robot
- Summary
- Q&A




# Introduction to ROS

- ROS is an open-source, flexible framework for writing robot software. It provides tools, libraries, and conventions to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.
- **Modularity:** Breaks down complex software into smaller, reusable components.
- **Communication:** Facilitates message passing between different parts of the robot's software.
- **Tool Support:** Offers a wide range of tools for simulation, visualization, and debugging.
- **Platform Agnostic:** Can run on various operating systems, although it is primarily used on Unix-based systems like Linux.
- **Flexibility:** Supports both academic research and industrial applications due to its modular and scalable architecture.

# ROS 1 vs ROS 2

## ROS 1:

- *Overview: The original version of ROS, widely adopted in academic and research environments.*
- *Strengths: Mature ecosystem, extensive package library, strong community support.*
- *Limitations: Limited support for real-time applications, single-point failure in the master node, less secure.*

## ROS 2:

- *Overview: A significant upgrade designed to address the limitations of ROS 1, built on the Data Distribution Service (DDS) standard.*
- *Real-Time Support: Improved support for real-time systems.*
- *Security: Enhanced security features.*
- *Distributed Architecture: No single-point failure, better suited for multi-robot systems.*
- *Improved Performance: More efficient communication and processing.*

# ROS Architecture

**Nodes:**

- *Nodes are individual processes that perform computation. In a ROS-based system, nodes communicate with each other to form a complete system.*
- *A node can be responsible for sensor data processing, another for control algorithms, and another for motor actuation.*

**Topics:**

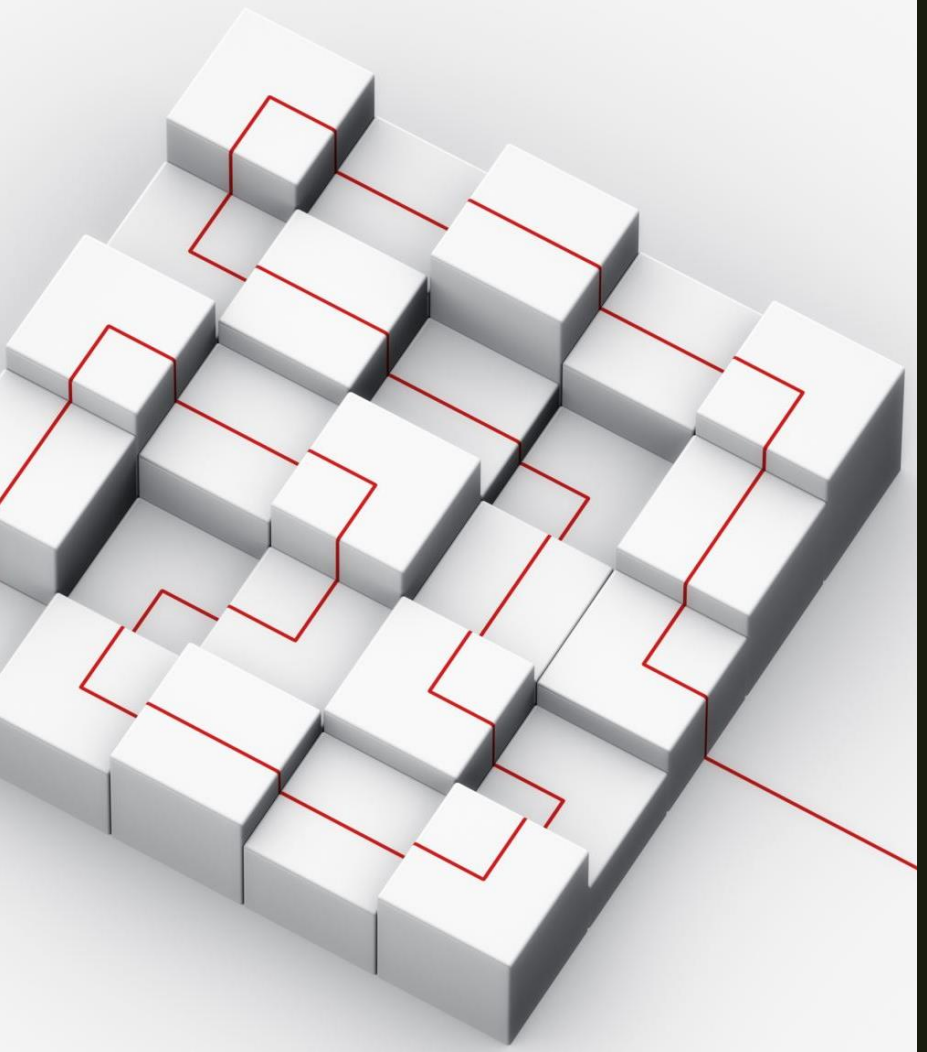
- *Topics are named buses over which nodes exchange messages. Nodes can publish messages to a topic or subscribe to a topic to receive messages.*
- *A sensor node publishes data to a topic named "/sensor_data," and a processing node subscribes to this topic to receive the data.*

**Services:**

- *Services allow for synchronous communication between nodes, enabling request-response interactions.*
- *A node can request the current state of a robot arm via a service call.*

**Parameter Server:**

- *A shared, multi-variate dictionary accessible via network APIs. It is used for storing configuration parameters.*
- *Storing calibration parameters for sensors.*



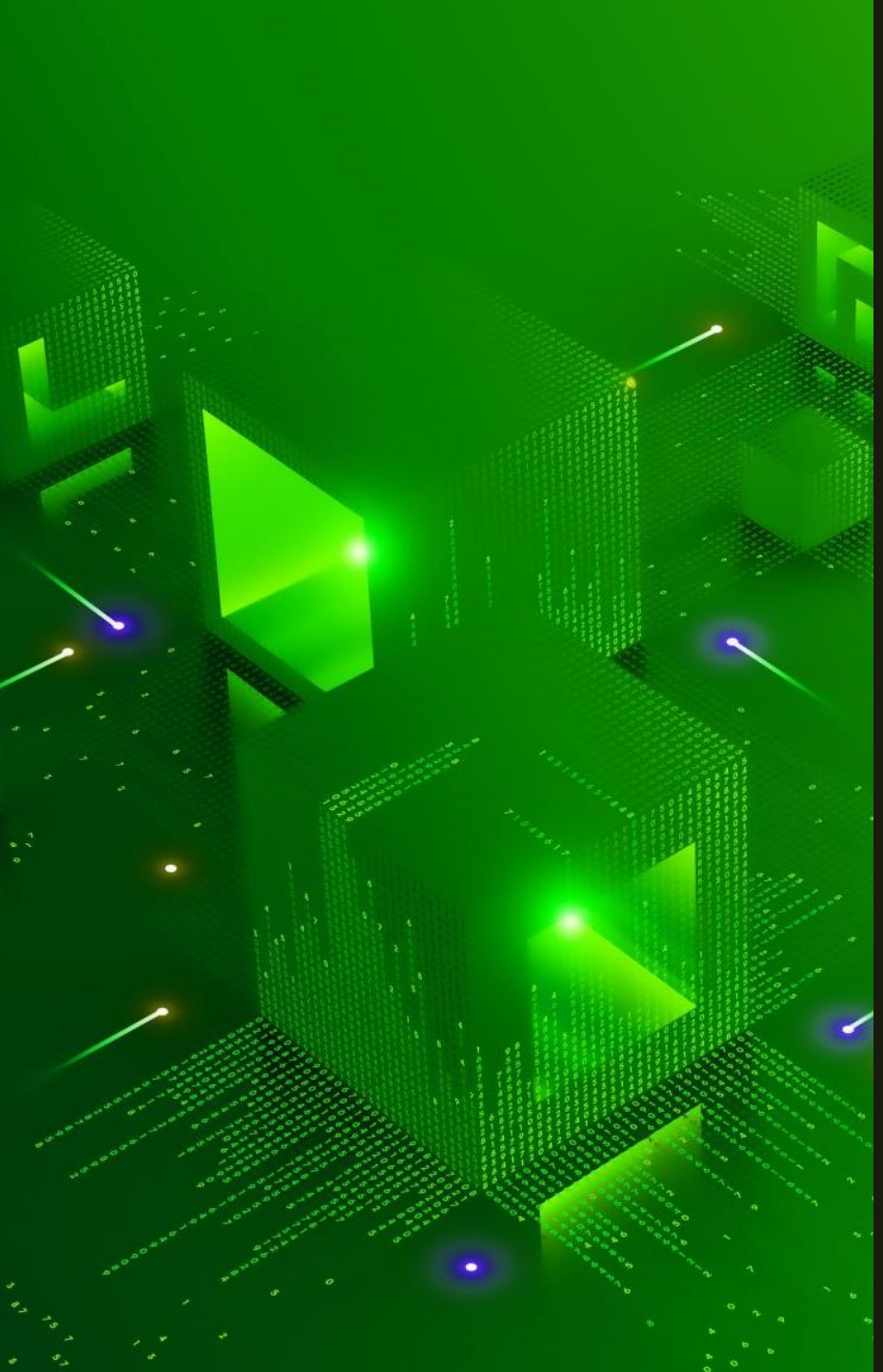
# Setting Up ROS

**Installation:**

- *Supported Platforms: Primarily supports Ubuntu Linux, but also compatible with other Unix-based systems.*
- *Steps: Detailed steps for installing ROS on Ubuntu, including adding the ROS repository, setting up keys, and installing ROS packages.*

**Workspace Setup:**

- *A workspace is a directory containing ROS packages and configurations.*
- *Steps: Creating a workspace, initializing it, and building the packages.*



# ROS Packages and Tools

**Packages:**

- *Packages are the primary unit of software organization in ROS. Each package can contain libraries, nodes, datasets, configuration files, and more.*
- *Example: turtlebot3, ros_control, and move_base.*

**Key Tools:**

- *roscore: Initializes the ROS master and essential services.*
- *rviz: A 3D visualization tool for visualizing sensor data and state information from ROS.*
- *rosbag: A tool for recording and playing back ROS message data.*
- *rqt: A graphical user interface framework for ROS, including plugins for visualization, introspection, and debugging.*

# ROS Communication

**Message Passing:**

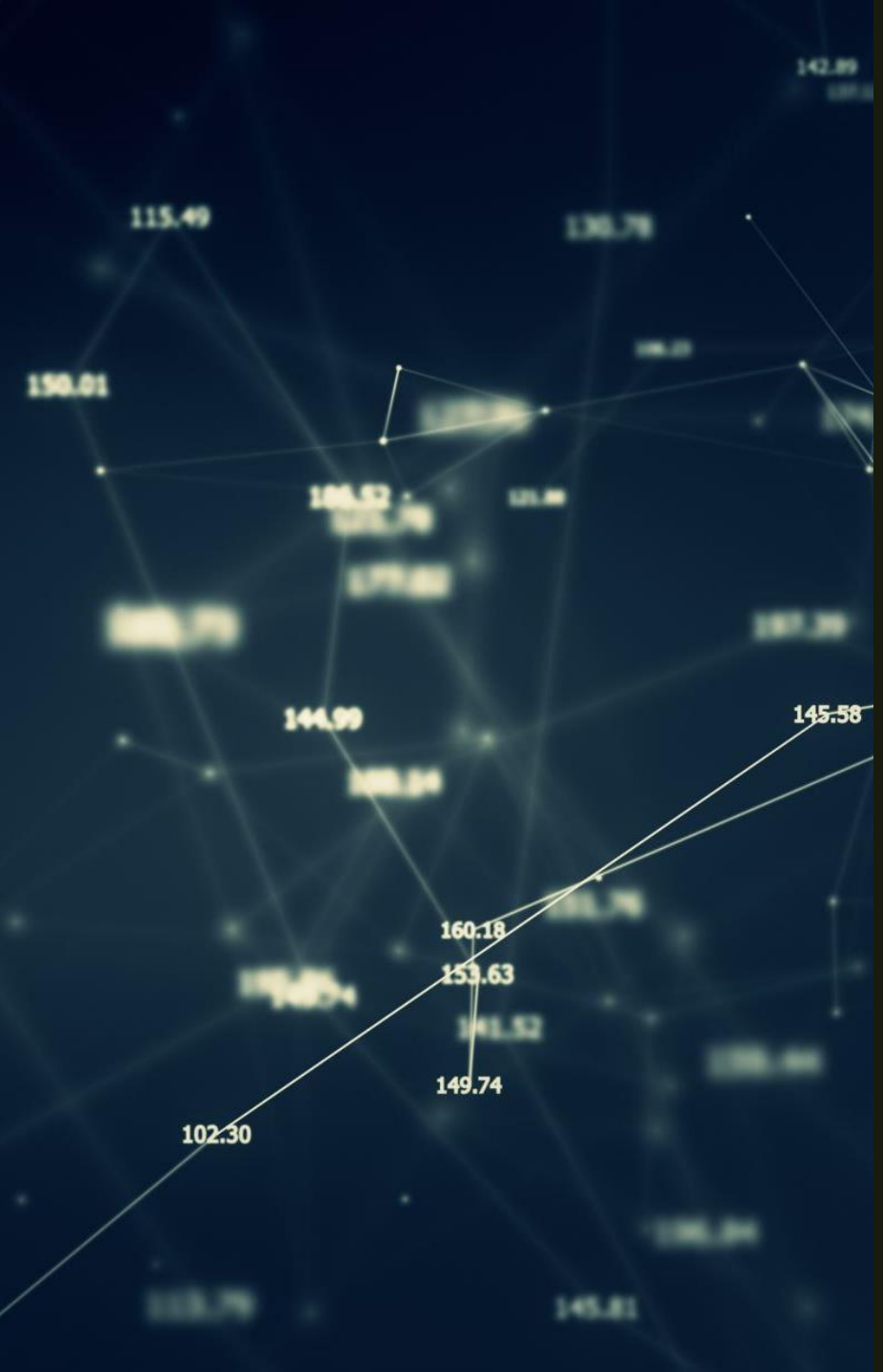
- *The primary method of communication in ROS, where nodes send and receive messages via topics.*
- *Example: A camera node publishes image data, while an image processing node subscribes to this data to perform analysis.*

**Services:**

- *Facilitate synchronous communication between nodes, allowing nodes to send a request and receive a response.*

**Actions:**

- *A higher-level communication mechanism that allows for asynchronous communication and feedback.*
- *Example: Used for tasks that take time to complete, such as moving a robot arm to a position.*




# ROS Messages

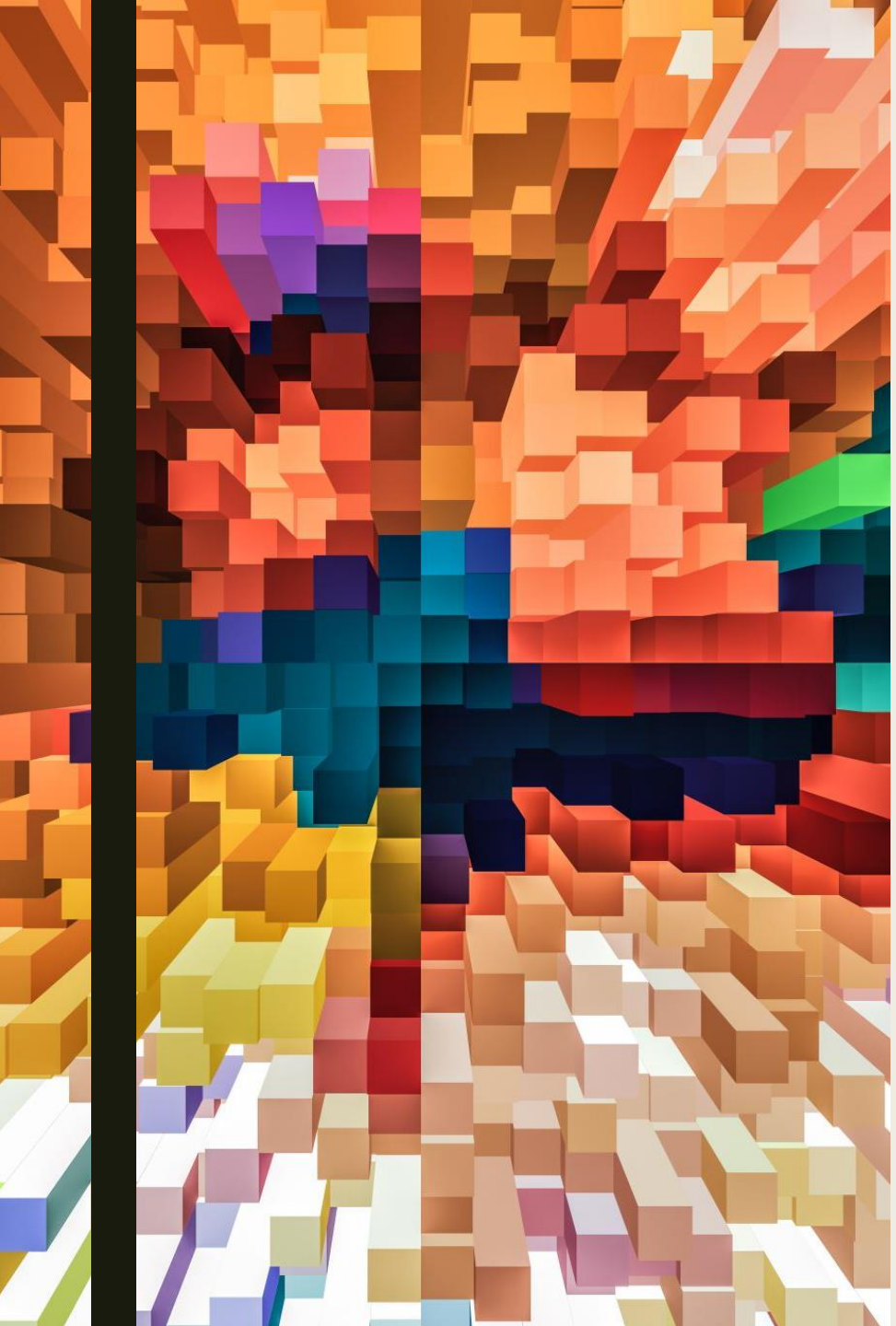
ROS messages are data structures used for communication between nodes. In ROS 2, they are defined using the same interface definition language (IDL) as ROS 1.

Creating a Custom Message:

> *Steps: Define the message in a .msg file, update the package configuration, and build the package.*

Usage:

- *Publishing: Broadcasting messages/data*
- *Subscribing: Receiving broadcasted messages/data*



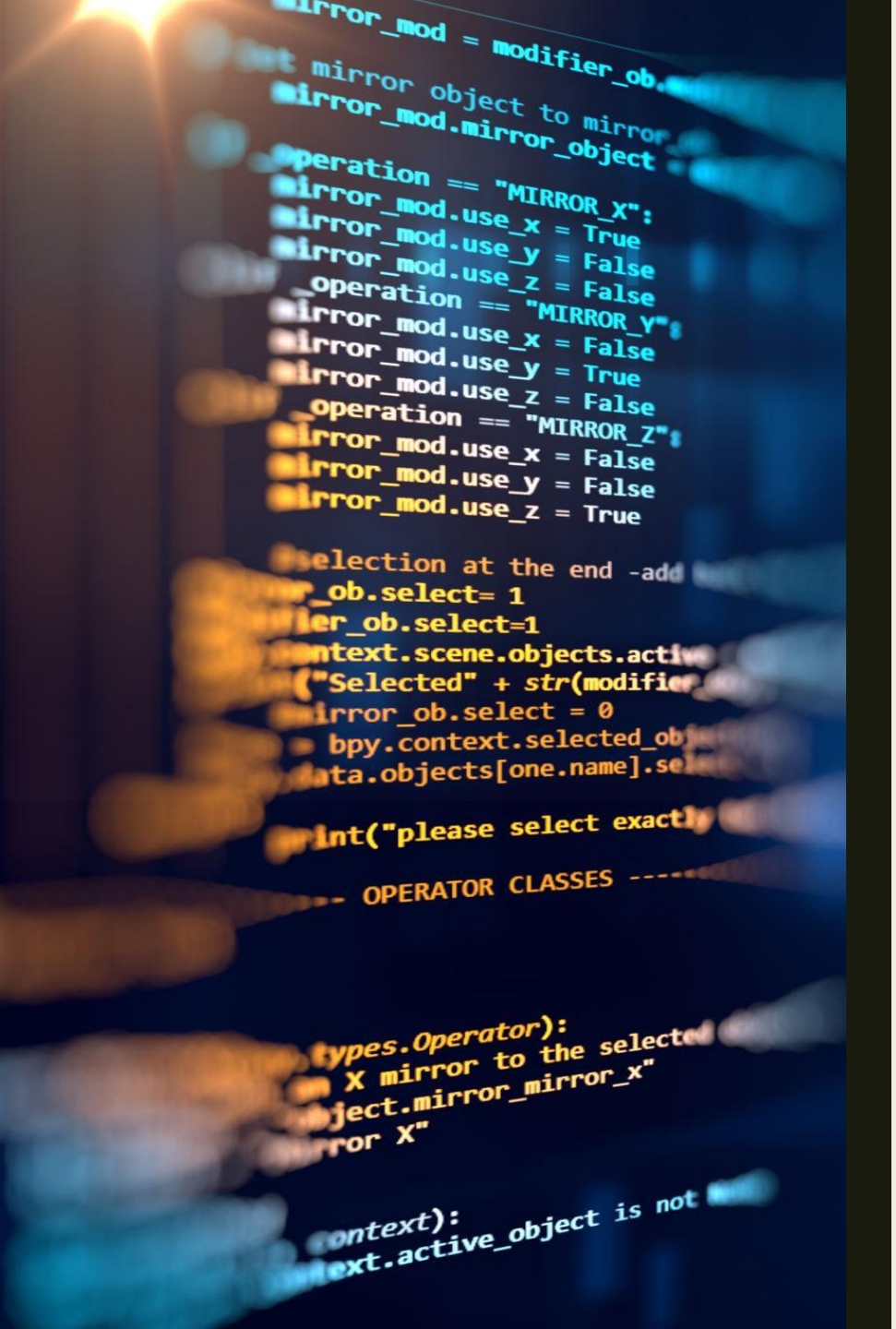
# Creating and Compiling ROS Nodes

**Creating a ROS 2 Node:**

- *Steps: Set up a ROS 2 workspace, create a package, and write a simple node.*

**Compiling the Node:**

- *Steps: Use colcon to build the package.*




# ROS Launch Files

Launch files are XML or Python scripts used to start multiple nodes and set their parameters.

**Creating a Launch File:**

- *Steps: Create a launch file using Python, specifying nodes and their configurations.*

**Running the Launch File:**

- *Steps: Use the ros2 launch command to execute the launch file.*

# Simulation in ROS

**Gazebo:**

- *A powerful robot simulation tool that integrates with ROS to provide realistic environments for testing and development.*
- *Features: Physics simulation, sensor simulation, and support for a wide range of robot models.*

**RViz:**

- *Primarily used for visualization, RViz can also be used to simulate and visualize sensor data and robot states.*
- *Features: Customizable displays for different types of data, interactive markers, and plugins for various sensors.*

# Building a Simple ROS Robot

```python
import rospy
from sensor_msgs.msg import LaserScan

def scan_callback(msg):
    # Process scan data
    pass

rospy.init_node('laser_scan_processor')
rospy.Subscriber('/scan', LaserScan, scan_callback)
rospy.spin()
```

Develop a simple mobile robot that can navigate and avoid obstacles using ROS.

**Components:**

- *Hardware: A basic robot chassis, motors, sensors (e.g., LIDAR or ultrasonic sensors).*
- *Software:*
  - Nodes: Create nodes for sensor data processing, navigation, and motor control.
  - Topics: Set up topics for publishing sensor data and subscribing to control commands.
- *rviz: Use for visualizing the robot's environment and state.*

- Sensor Node (Python):

# Summary

- We covered an overview of ROS and its important tools and features.
- Check ROS office website for more information: ros.org
- Refer to online resources for further reading. For example, this website provides good details: https://www.theconstruct.ai/.

# Q&A